ОТЧЁТ

по практическому заданию

«Мониторинг API и контейнеризация сервиса»

Проект infra-monitoring-project

Аккаунт GitHub: chavonadelal

2026

1 Организация проекта

Цель работы - закрепить навыки написания Bash-скриптов, анализа сетевого трафика, работы с Git, настройки CI/CD и использования Docker Compose.

Создан публичный репозиторий infra-monitoring-project. Начальные файлы README.md, .gitignore и .gitattributes добавлены в ветку main. Реализация выполнена в рабочей ветке feature/monitoring.

В .gitignore исключены логи, временные файлы и артефакты сборки. Для Bash-скриптов задан формат строк LF и установлен исполняемый бит.

Ветка main защищена: изменения вносятся через Pull Request после успешной проверки Bash checks. Создан Pull Request № 1 с описанием изменений и результатов проверок. Он оставлен открытым для проверки.

Ссылка на репозиторий:
https://github.com/chavonadelal/infra-monitoring-project

Ссылка на Pull Request:
https://github.com/chavonadelal/infra-monitoring-project/pull/1

Работа выполнена в Windows. Для запуска скриптов использован Git Bash, для контейнеров - Docker Desktop с Linux-контейнерами. Для просмотра сетевого дампа использован Wireshark 4.6.9.

2 Анализ сетевого трафика

Выполнены HTTP-запросы к публичному API httpbin.org. Полученный трафик сохранён в файл http-traffic.pcapng и проанализирован в Wireshark с фильтром http (рисунок 1).

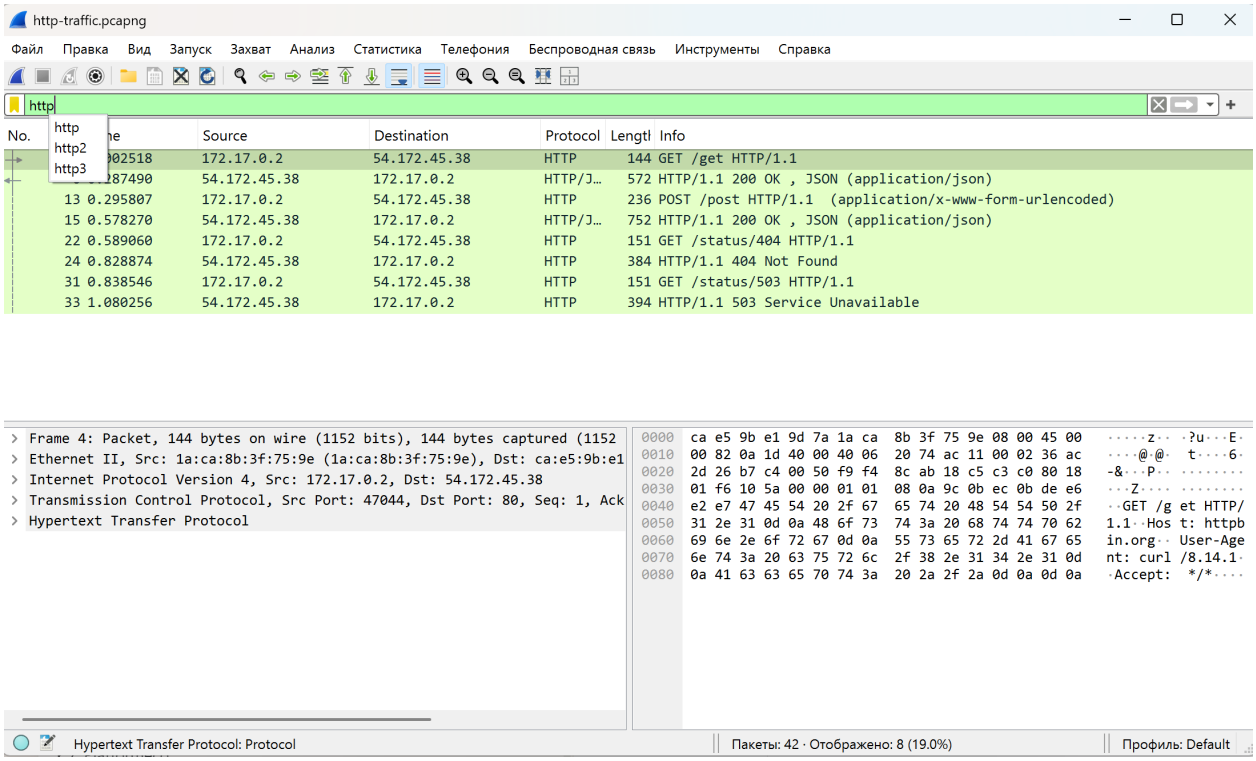


Рисунок 1 - HTTP-трафик в Wireshark

В обмене участвовали клиент 172.17.0.2 и сервер 54.172.45.38:80.

Использованы методы GET и POST. На GET /get и POST /post получен код 200 OK, на GET /status/404 - 404 Not Found, на GET /status/503 - 503 Service Unavailable.

В дампе 42 пакета, из них 8 HTTP-пакетов. Файл http-traffic.pcapng сохранён в docs/evidence и вложен в этот PDF. Для просмотра методов и кодов использован HTTP: в HTTPS эти данные зашифрованы.

3 Bash-скрипты

3.1 Проверка доступности API

Скрипт check_api.sh принимает URL и выполняет запрос через curl. При HTTP 200 выводится «API доступен», код завершения равен 0. При другом HTTP-коде, неверном URL или сетевой ошибке возвращается 1.

Ниже приведён фактический вывод при проверке доступных адресов, адреса с HTTP 404 и невалидного URL.

```
$ ./check_api.sh http://localhost:80
API доступен
exit code: 0

$ ./check_api.sh https://httpbin.org/status/200
API доступен
exit code: 0

$ ./check_api.sh https://httpbin.org/status/404
API недоступен, код ответа: 404
exit code: 1

$ ./check_api.sh not-a-url
Использование: ./check_api.sh http[s]://адрес/api
API недоступен, код ответа: 000 (некорректный URL)
exit code: 1
```

3.2 Анализ логов

Скрипт analyze_logs.sh создаёт тестовый access.log из 360 строк. Для разбора использованы grep, awk, sed, sort и uniq. Регулярное выражение проверяет IPv4-адреса с октетами от 0 до 255.

Результат запуска анализатора:

```
$ ./analyze_logs.sh --generate
Создан тестовый файл: access.log (360 строк)
Общее количество запросов: 360
Топ-3 IP-адреса (запросы IP):
120 192.168.1.10
90 10.0.0.5
60 172.16.0.3
Запросов с кодами 4xx: 72
Запросов с кодами 5xx: 72
exit code: 0
```

```
$ wc -l access.log
360 access.log
```

Получено 360 запросов. Три наиболее активных IP-адреса выполнили 120, 90 и 60 запросов соответственно. С кодами 4xx найдено 72 запроса, с кодами 5xx - также 72.

Оба скрипта сделаны исполняемыми командой chmod +x. Проверка bash -n не выявила ошибок синтаксиса. Результаты запуска check_api.sh приведены в подразделе 3.1, результат analyze_logs.sh - выше.

4 Docker Compose

В файле docker-compose.yml описаны два сервиса. Сервис api использует готовый образ hashicorp/http-echo:1.0.0, слушает порт 8080 и возвращает строку Monitoring Service Active. Сервис proxy использует nginx:alpine и перенаправляет запросы с порта 80 на api:8080.

Для связи контейнеров создана отдельная bridge-сеть monitoring. Конфигурация Nginx подключена в режиме чтения. Healthcheck проверяет доступность API через прокси и текст ответа.

Контейнеры запущены командой docker compose up -d --wait. На рисунке 2 показан вывод docker-compose ps с выбранными столбцами Service, Image и Status, а также проверка ответа API.

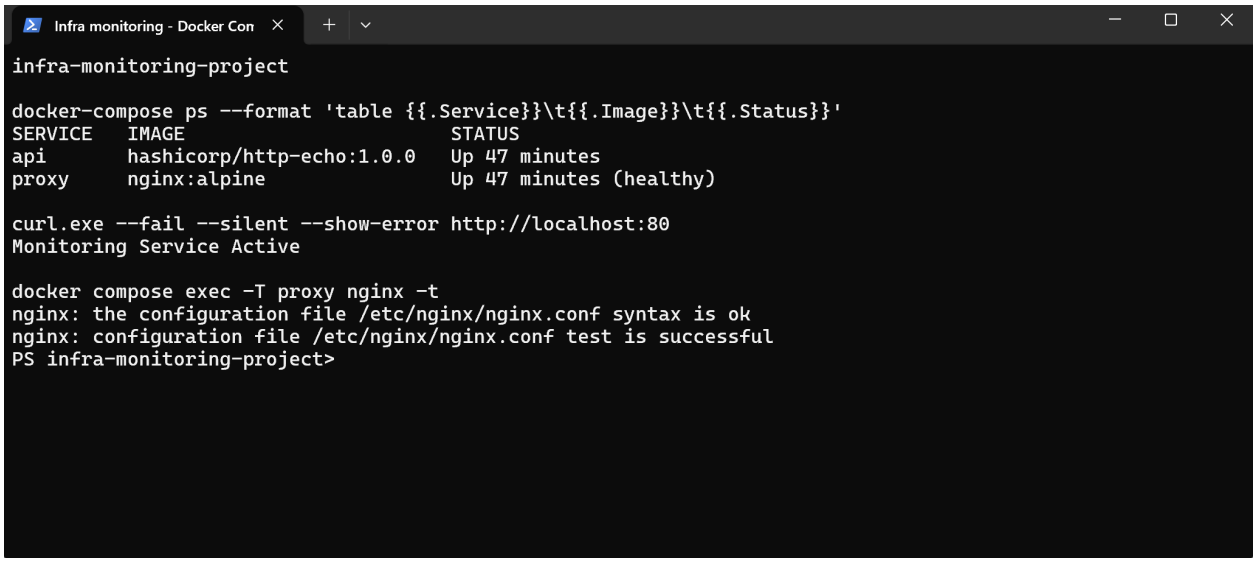


Рисунок 2 - Работающие контейнеры Docker Compose

Оба контейнера находятся в состоянии Up, прокси имеет статус healthy. Запрос curl к http://localhost:80 возвращает Monitoring Service Active. Проверка nginx -t завершилась успешно. Полный текстовый вывод команды сохранён в docs/evidence/docker-output.txt.

5 CI/CD в GitHub Actions

В .github/workflows/ci.yml настроен workflow, который запускается при создании и обновлении Pull Request в main. Выполняются проверки из задания:

bash -n check_api.sh
bash -n analyze_logs.sh
./check_api.sh https://httpbin.org/status/200

Все команды завершились с кодом 0. Скриншот успешного запуска проверки Bash checks приведён на рисунке 3.

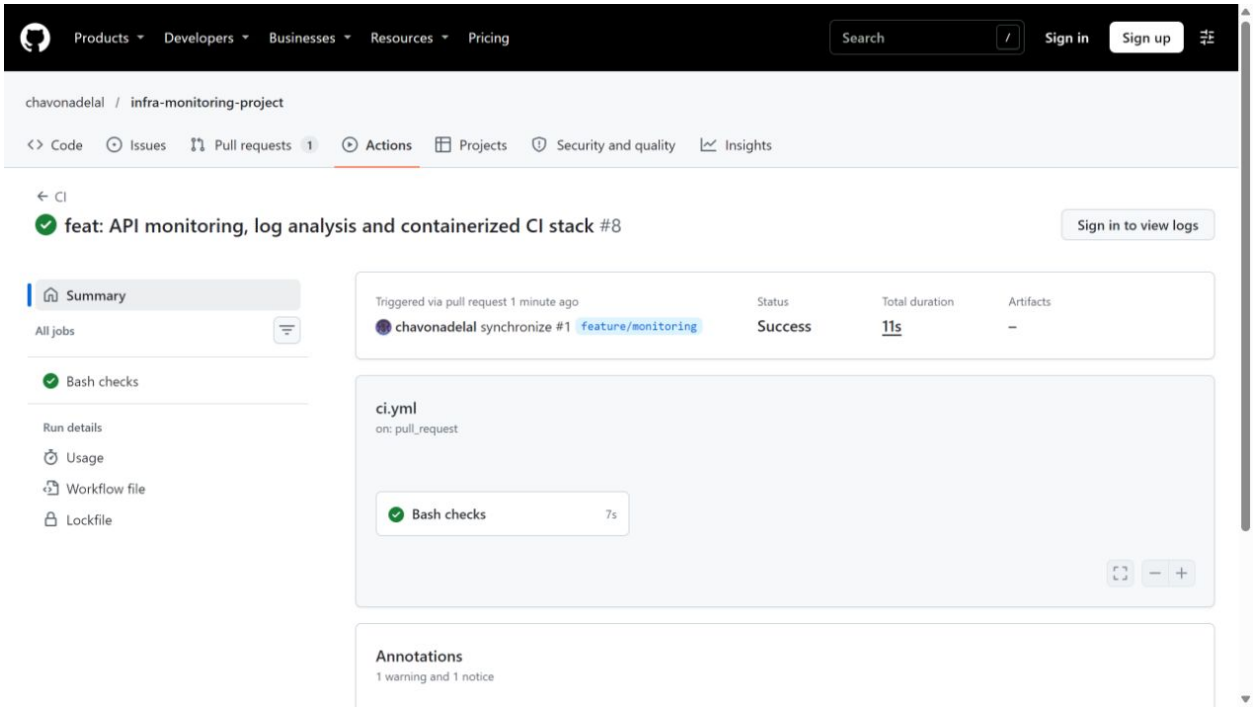


Рисунок 3 - Успешный запуск GitHub Actions

Ссылка на запуск:
Успешный запуск CI в GitHub Actions

6 SSH и SCP

В README.md описана передача проекта на удалённый сервер. Для подключения создаётся ключ Ed25519 с парольной фразой. Публичный ключ устанавливается на сервер командой ssh-copy-id. Приватный ключ в репозиторий не добавляется.

Пример команд (адрес deploy@example.org необходимо заменить своим):

```
ssh-keygen -t ed25519 -a 100 -f ~/.ssh/infra_monitoring
ssh-copy-id -i ~/.ssh/infra_monitoring.pub deploy@example.org
git archive --format=tar.gz -o ../project.tar.gz HEAD
scp -i ~/.ssh/infra_monitoring ../project.tar.gz deploy@example.org:~/
ssh -i ~/.ssh/infra_monitoring deploy@example.org
```

После подключения архив распаковывается в каталог проекта, затем выполняются docker compose up -d --wait и проверка curl http://localhost:80. Полный пример развёртывания приведён в README.md.

При первом подключении проверяется отпечаток ключа сервера. Проверка ключа хоста не отключается. В README описаны генерация SSH-ключей, установка публичного ключа, передача архива через SCP и команды развёртывания проекта.